

Big Data Analytics

Big Data Processing and AI-Powered Search for Laptop Recommendations

Table of Contents

1. Introduction	3
2. Methodology and Data Preprocessing	3
2.1 Data Collection and Sources	3
2.2 Data Cleaning and Normalization	3
2.3 LLM-Assisted Data Processing	3
2.4 Big Data Processing and RAG Pipeline	4
2.5 Evaluation Methodology	4
2.6 Data Preprocessing Summary	4
3. Dataset Characteristics and Exploratory Data Analysis	5
4. Feature Engineering	5
5. Big Data Analytics Pipeline & RAG Chatbot Implementation	5
5.1 Spark Data Processing Pipeline	6
5.2 Distributed Embedding Generation	6
5.3 Vector Similarity Search (FAISS)	6
5.4 RAG Chatbot Backend & Smoke Testing	6
5.5 Retrieval Evaluation	7
5.6 Gradio Frontend & Interactive Querying	7
5.7 Qualitative (LLM-as-Judge) Evaluation	8
6. Discussion	8
7. Conclusion	8
Appendix: Data Sourcing	9

1. Introduction

Laptop pricing in emerging e-commerce markets exhibits significant variance due to heterogeneous hardware specifications, inconsistent retailer data reporting practices, and complex market segmentation. Bangladesh, with a rapidly growing middle class and increasing e-commerce adoption, presents a dynamic context for investigating price prediction and product-discovery mechanisms.

This project develops a two-part system for the Bangladesh laptop e-commerce market: a machine learning framework for predicting laptop prices from a curated dataset of 2,460 laptop listings (10 raw fields spanning product names, pricing, hardware specifications, and market indicators), and a big-data retrieval-augmented generation (RAG) pipeline built on the same product catalogue.

The big-data pipeline is implemented end-to-end in Apache Spark: the raw 2,460-listing catalogue is cleaned and normalized via the Spark DataFrame API, checked for exact duplicates (none found on the name/price/cpu/ram/storage key), and passed through MinHash and Locality-Sensitive Hashing (LSH) to resolve 403 near-duplicate clusters, leaving 2,057 unique products. Product text is split into 3,433 overlapping chunks via Spark mapPartitions and embedded into 384-dimensional vectors with the all-MiniLM-L6-v2 sentence-transformer, distributed across partitions using a pandas_udf. The resulting vectors are indexed with FAISS — IndexFlatL2, IndexIVFFlat, and IndexHNSWFlat were benchmarked, with IndexFlatL2 selected for serving because it gives exact recall at negligible latency cost on this corpus size — and served through a FastAPI backend that embeds each query, retrieves matching chunks with optional price, brand, and category filters, and generates grounded recommendations with a Groq-hosted LLM exposed through a Gradio chat interface.

Retrieval quality was measured with Precision@k and Recall@k against a hand-labeled set of 31 queries (evaluation_queries.csv); 5 of these referenced product IDs no longer present after deduplication, so the evaluation fell back to 30 synthetic queries generated directly from the current serving metadata, each with a verifiably correct expected match. Across all three FAISS index types the pipeline reached 63.3% Precision@1, 90.0% Recall@3, and 93.3% Recall@5, with the approximate indexes (IVF, HNSW) matching IndexFlatL2's precision and recall while returning results 2.5–4× faster.

2. Methodology and Data Preprocessing

2.1 Data Collection and Sources

Laptop data were collected from four primary sources: Ryans Computers, Global Brand PLC, Dazzle, and an additional laptop price dataset exported from Gigasheet. For the web-based sources, an automated scraping workflow was used to identify laptop listing pages, traverse available pagination, collect product URLs, and extract product prices, descriptions, and specifications.

2.2 Data Cleaning and Normalization

The collected records were then consolidated in stages (see Appendix: Data Sourcing for the full breakdown). The 2,000 records from the Ryans Computers Excel dataset were merged with 166 web-scraped records from Global Brand and Dazzle, producing 2,166 records. A quality and completeness assessment was then performed on this merged set, retaining the 850 most suitable records and removing 1,316 incomplete or lower-quality records. These 850 records were merged with an additional 2,160 records from the Gigasheet laptop price dataset, resulting in 3,010 records. Records without a usable price were subsequently removed, leaving the final 2,460-record dataset, which was restricted to the required attributes plus product name. As a final normalization step, price values below 10,000 were multiplied by 120 to correct for inconsistent price representation across sources.

2.3 LLM-Assisted Data Processing

Price values were normalized across common representations such as BDT, Tk, and ₹, and records without valid numeric prices were removed. Hardware specifications were also standardized; for example, RAM and storage values expressed in mixed formats such as “16GB,” “512GB SSD,” and “1TB” were converted into consistent numerical representations. Missing GPU and battery information was retained using missingness indicators rather than replacing unavailable information with fabricated values.

Before modeling, all source datasets were converted into a common schema to address differences in column names, formats, and hardware representations. The main structured attributes were price, CPU, RAM, storage, GPU, display, battery, and category tags, while product descriptions and specification text were retained as a combined unstructured-text field.

2.4 Big Data Processing and RAG Pipeline

After requiring usable price, RAM, storage, and display values and removing the extreme 0.5% price tail, the modeling dataset consisted of 2,286 records and 27 engineered features.

Feature engineering was performed to convert noisy and heterogeneous laptop information into machine-learning-ready variables. RAM, storage, and display information were reconstructed primarily from product names and specification text using pattern matching, recovering usable values for 2,310 of the 2,460 records. CPU specifications were transformed into hierarchical performance tiers, while storage type, display size, touchscreen, gaming, business, and ultrabook indicators were derived from the available product information. Brand names were extracted and encoded, and GPU and battery missingness indicators were incorporated.

2.5 Evaluation Methodology

Finally, the retrieval system was integrated with a Groq-based LLM through a FastAPI backend. User queries are embedded using the same embedding model, relevant products are retrieved through FAISS with optional price, brand, and category filters, and the retrieved context is supplied to the LLM to generate grounded product recommendations. A Gradio interface was used to provide interactive natural-language querying.

The text chunks were converted into 384-dimensional vector embeddings using the all-MiniLM-L6-v2 model through distributed Spark processing. These embeddings were indexed using FAISS to support efficient semantic similarity search. Three FAISS index types—IndexFlatL2, IndexIVFFlat, and IndexHNSWFlat—were benchmarked, with IndexFlatL2 selected for serving because it provided exact recall while remaining sufficiently fast for the relatively small corpus.

For product discovery, the processed dataset was further analyzed using an Apache Spark-based pipeline. Text normalization and exact duplicate checking were performed first, followed by MinHash and Locality-Sensitive Hashing (LSH) for near-duplicate detection. This reduced the 2,460 records to 2,057 unique products. The product text was then divided into overlapping chunks, producing 3,433 text segments.

2.6 Data Preprocessing Summary

Data preprocessing involved multiple stages to prepare raw laptop records for modeling:

- **Price Filtering & Validation:** Removal of records with missing or non-numeric prices, resulting in 2,460 valid observations from the original 3,010 merged records.
- **Format Normalization:** Standardization of hardware specifications from mixed representations to uniform numeric formats. RAM values recorded as “16” and “16GB” were parsed to numeric GB. Storage values represented as “512 SSD”, “512GB SSD”, or “1TB” were standardized to numeric GB capacity.
- **Missing Value Handling:** Fields with minimal missingness (price, CPU, display) were retained as complete. GPU (55.7% missing) and battery (97.5% missing) were retained with indicator variables capturing data absence, distinguishing between true absence and data unavailability.
- **Category Multi-Label Processing:** The category_tags field contained overlapping labels (“Touchscreen”, “Business, Touchscreen”), indicating multi-label rather than single-category encoding, requiring specialized handling.
- **Text Price Leakage Mitigation:** The combined_unstructured_text field contained explicit price mentions (e.g., “starting from 73,900 BDT”); these are excluded from the numeric feature set used for price prediction.

- Feature Scaling: Numerical features were standardized to zero mean and unit variance for the linear model family (Linear/Ridge/Lasso); tree-based models (Random Forest, XGBoost) used unscaled features.

3. Dataset Characteristics and Exploratory Data Analysis

The final dataset contains 2,460 laptop records with 10 raw fields spanning product names, pricing, hardware specifications, and market indicators.

Price Distribution: Prices follow a right-skewed distribution with Mean: 159,700 BDT, Median: 124,883 BDT, Std. Dev.: 111,006 BDT, Min: 24,126 BDT, Max: 858,056 BDT. This range spans entry-level through premium segments.

CPU Diversity: 189 unique CPU variants with dominant tiers being Intel Core i7 (28.9%), Intel Core i5 (21.8%), AMD Ryzen 7 (6.3%), Intel Core i3 (5.3%), and AMD Ryzen 5 (5.2%).

RAM and Storage Concentration: RAM clusters at 16GB (37.7%) and 8GB (33.2%). Storage concentrates at 512GB SSD (38.0%) and 1TB SSD (23.1%).

Sparse GPU and Battery Data: GPU is reported directly in the gpu column for 44.3% of records (55.7% missing); battery information is present in only 2.5% of records (97.5% missing), reflecting inconsistent retailer data practices.

Categorical Distribution: Multi-label tags show Touchscreen appearing in 53.7% of records, with 10.0% having “Business, Touchscreen” and 9.8% having “Gaming, Touchscreen”.

4. Feature Engineering

Effective feature representation is essential for regression model performance. Because RAM, storage and display size are frequently missing or buried in long free-text spec strings in the dedicated ram/storage/display columns, these fields were reconstructed primarily by pattern-matching the structured product_name field (e.g., “...Intel Core i7-1165G7/16GB/512GB SSD/15.6””), recovering usable values for 2,310 of 2,460 records (93.9%).

- Hierarchical CPU Tier Encoding: CPU text aggregated into ordinal performance tiers (tier 4: i9/Ryzen 9; tier 3: i7/Ryzen 7; tier 2: i5/Core 5/Ryzen 5; tier 1: i3/Celeron/Pentium/Ryzen 3; tier 0: unspecified), capturing non-linear step-wise price increases across processor families.
- Hardware Attribute Standardization: RAM (GB) and storage (GB, with TB→GB conversion) parsed from mixed formats; storage type flag (SSD / HDD / eMMC / Unknown) captures performance characteristics.
- Display and Form Factor Features: Display diagonal (inches) extracted via regex from product name / display text. Binary touchscreen, gaming, business and ultrabook indicators created from category_tags presence.
- Brand Extraction and Encoding: Brand names extracted from product names and one-hot encoded for major brands (Asus, Lenovo, Dell, HP, MSI, Acer, Apple, Microsoft, Razer, Samsung, and others), enabling brand premium effect estimation.
- GPU Features: Binary GPU-present indicator plus a dedicated-GPU flag (RTX/GTX/Radeon RX/Arc series detected in either the gpu field or the product name).
- Missing Value Indicators: Binary indicators for GPU and battery presence, distinguishing data availability from true absence.

After requiring usable price, RAM, storage and display values, and trimming the extreme 0.5% price tail (a handful of outlier workstation/server-class listings), the modeling dataset comprises 2,286 records across 27 engineered features (11 numeric/binary + 3 storage-type dummies + 13 brand dummies).

5. Big Data Analytics Pipeline & RAG Chatbot Implementation

This project implements a big data analytics pipeline combining Apache Spark for large-scale data processing, FAISS for efficient vector similarity search, and Groq for grounded natural language generation. The architecture processes laptop product listings through cleaning, deduplication, chunking, distributed embedding generation, and indexing for downstream retrieval-augmented generation (RAG). All figures below are taken directly from the executed notebook (CSE488_BigData_RAG_Pipeline_.ipynb) outputs, run end-to-end locally with TestClient (no live server/ports).

5.1 Spark Data Processing Pipeline

Input: 2,460 laptop product listings from the merged dataset (final_merged_dataset.csv).

- Text Normalization: Collapsed repeated whitespace, trimmed leading/trailing spaces on all text columns via the Spark DataFrame API.
- Exact Deduplication: 0 exact duplicates removed on the (name, price, cpu, ram, storage) key — all 2,460 rows were already unique on this key.
- Near-Duplicate Detection: MinHash (5-character shingles of a compact spec_text field) + LSH with Jaccard distance ≤ 0.15 .
- 584 near-duplicate pairs identified; 403 clusters collapsed to a single representative each.
- Final unique products: 2,057 (16.4% collapse rate from 2,460).
- Text Chunking: Sliding word window (chunk_size=120, overlap=20) over combined_unstructured_text via Spark mapPartitions, cross-checked against an equivalent UDF+explode implementation (identical chunk count).
- 2,057 products $\rightarrow$ 3,433 total chunks.

5.2 Distributed Embedding Generation

Model: sentence-transformers/all-MiniLM-L6-v2, loaded successfully in this run (no offline fallback was needed — the Lite TF-IDF+SVD embedder is implemented as a fallback path but was not triggered).

- Output Dimensionality: 384-dimensional dense vectors, L2-normalized.
- Spark Implementation: pandas_udf with batch_size=64 for Arrow-batched inference, coalesced to 4 partitions to cap concurrent model loads.
- Measured Processing Time: 278.3 seconds to embed all 3,433 chunks (≈ 0.081 s/chunk).
- Output: Parquet format with 3,433 rows (chunk_id, chunk_text, embedding, product metadata); round-trip read-back verified at 3,433 rows.

5.3 Vector Similarity Search (FAISS)

Three FAISS index types were built and benchmarked on the same 3,433 vectors (384-dim), using 200 random query vectors and recall@10 against IndexFlatL2 as ground truth:

Index Type	Recall@10	Avg Latency (ms/query)	Total Search Time (s)
IndexFlatL2 (baseline)	1.0000	0.204	0.0408
IndexIVFFlat (nlist=100, nprobe=8)	0.9945	0.030	0.0061
IndexHNSWFlat (M=32, efConstruction=80, efSearch=64)	1.0000	0.043	0.0086

Decision Rationale: IndexFlatL2 was chosen for serving. At this corpus size (3.4k vectors) exact brute-force search is already sub-millisecond, so there is no practical latency penalty for guaranteeing exact recall; IVF and HNSW happened to benchmark faster in wall-clock terms on this small corpus (their speed advantage normally shows up at 100k+ vectors, where FlatL2 becomes the bottleneck), but IVF's 99.45% recall means occasional missed neighbors that FlatL2 avoids entirely.

5.4 RAG Chatbot Backend & Smoke Testing

Architecture: FAISS IndexFlatL2 loaded from Parquet (3,433 indexed chunks) → runtime query embedded with the same all-MiniLM-L6-v2 model → top-k semantic search with optional metadata filters (price range, brand, category) → Groq LLM (openai/gpt-oss-120b) generation grounded in retrieved context → FastAPI /chat endpoint with structured JSON response, exercised via TestClient.

Smoke test queries and outcomes (live Groq generation, groq_rag mode):

- “Gaming laptop with dedicated GPU under 150,000 BDT” → returned 3 Lenovo LOQ Gaming configurations, all with dedicated RTX GPUs and prices between 101,900–144,000 BDT (all within the stated budget).
- “Laptop for programming with 16GB RAM” (60k–150k BDT filter) → returned 3 laptops, all 16GB RAM, prices 113,500–148,000 BDT (all within range).
- “Gaming laptop under 200,000 BDT” → returned 2 gaming laptops with dedicated GPUs, both under budget.

All smoke-test recommendations respected the stated price/spec constraints and cited match scores and L2 distances for each retrieved chunk.

5.5 Retrieval Evaluation

A hand-labeled evaluation_queries.csv (31 rows) was supplied, but 5 of its expected_product_id values were not present in the current serving metadata (the dataset/deduplication had changed since the file was labeled). The notebook detected this mismatch and fell back to generating 30 synthetic evaluation queries directly from the current serving metadata, so all 30 queries have a verifiably-correct expected match. This is a data-consistency issue worth flagging: the original evaluation_queries.csv should be regenerated against the current deduplicated product set if it is to be used for evaluation going forward.

Precision@k / Recall@k across the 30-query set, for all three index types:

Index	k	Precision	Recall	Avg Latency (ms)	Avg Match Score
flat	1	63.3%	63.3%	0.547	94.90
flat	3	30.0%	90.0%	0.552	94.15
flat	5	18.7%	93.3%	0.805	93.60
hns w	1	63.3%	63.3%	0.212	94.90
hns w	3	30.0%	90.0%	0.211	94.15
hns w	5	18.7%	93.3%	0.220	93.60
ivf	1	63.3%	63.3%	0.163	94.90
ivf	3	30.0%	90.0%	0.142	94.15
ivf	5	18.7%	93.3%	0.155	93.60

Interpretation: precision and recall are identical across index types at every k — on this corpus size, the approximate indexes (IVF/HNSW) return the same product-level neighbors as exact search, they just do it 2.5–4× faster.

Precision@1 = 63.3% (correct product is the single top hit for 19/30 queries); recall reaches 93.3% by k=5 (28/30 queries have the correct product somewhere in the top 5). k=3 is a reasonable serving default, balancing a compact 3-recommendation UI against 90% recall.

5.6 Gradio Frontend & Interactive Querying

A Gradio web application was launched (public share link generated, valid up to one week) with the following components:

- Input: natural-language query textbox, price range filters (min/max BDT), brand filter, category filter, top-k slider (1–5).
- Output: Groq-generated grounded answer, structured product recommendations (ID, name, price, key specs, match explanation), and the retrieved evidence chunks that informed the answer.

A final structural verification cell confirmed: embedding dimension consistency, vector/chunk count parity, all three FAISS indexes present, ≥30 evaluation queries, k∈{1,3,5} evaluated, the live Groq smoke test succeeded, and match scores were present on every retrieved hit and evaluation row.

5.7 Qualitative (LLM-as-Judge) Evaluation

Beyond retrieval metrics, the 31 evaluation queries were re-run through the live /chat endpoint and the resulting answers were judged by the Groq LLM itself (acting as an automated evaluator) rather than a human rater, split into four batches of up to 8 cases to keep each judging request within Groq's free-tier token limits. Each case was scored on four binary criteria: groundedness (does every recommended product_id actually appear in the retrieved evidence set), correctness of the recommendation, presence of unsupported claims, and usefulness of the stated reasoning.

Three of the four judging batches completed successfully; one batch (covering roughly a third of the queries) failed Groq's JSON validation step and fell back to a "not_run" placeholder for that subset rather than a fabricated score. For the batches that completed, results were consistently strong: sampled outputs showed the answers were grounded and correct with no unsupported claims and useful reasoning, consistent with the strict, evidence-only prompt used for generation in Section 5.4.

Limitation: because one judging batch did not complete, the qualitative results above should be read as a partial, indicative check rather than a full 31-query audit; re-running the failed batch (or increasing max_tokens/simplifying the response schema) would be needed for complete coverage.

6. Discussion

- Greater price variance in equivalent configurations across retailers.
- Less standardized hardware specification reporting compared to developed markets — recovering RAM/storage/display required regex parsing of free-text product names because the dedicated structured columns were themselves inconsistently populated.
- Smaller transaction volumes enabling higher price dispersion.
- Concentrated rather than uniform brand effects, reflecting limited consumer information availability for a handful of premium/imported brands.
- Retailer-specific data practices (e.g., near-total absence of battery specs) directly limiting which features a model can use.
- Operational/security limitation: a since-deprecated Groq API key was found hard-coded as a fallback default in the notebook's client-configuration cell despite an accompanying comment stating it had been removed — a reminder to grep for literal key strings before sharing or committing notebooks, even after a key is believed to be pulled out into an environment variable.

7. Conclusion

This project develops an end-to-end big data pipeline for intelligent laptop product discovery in Bangladesh's e-commerce market. Key results:

- Construction of a curated 2,460-record dataset with systematic quality assessment; 2,286 records retained for price modeling after requiring complete RAM/storage/display specs.
- Domain-specific feature engineering recovering hardware specs from noisy free text, capturing hierarchical CPU tiers, storage type, brand, and category signals.
- A full Spark → FAISS → RAG pipeline: 2,460 listings deduplicated to 2,057 unique products (MinHash+LSH, 16.4% collapse rate), chunked into 3,433 segments, embedded with all-MiniLM-L6-v2 (384-dim) via distributed pandas_udf, and indexed across three FAISS variants.
- A working RAG chatbot (FastAPI + Groq + Gradio) achieving 90% recall@3 and 63.3% precision@1 on a 30-query evaluation set, with all smoke-test recommendations respecting stated price and spec constraints.

Future work should explore temporal price dynamics, market segmentation analysis across buyer segments (gaming, business, ultrabook), hybrid retrieval combining semantic search with keyword/filter-based ranking, and end-to-end production deployment with real-time index updates.

Appendix: Data Sourcing

Step	Dataset / Operation	Records Before	Records After
1	Ryans Computers Excel dataset	2,000	2,000
2	Global Brand + Dazzle web-scraped data	166	166
3	Merge Ryans + web-scraped data	2,166	2,166
4	Quality/completeness ranking and selection	2,166	850
5	Additional attached laptop dataset	2,160	2,160
6	Merge selected 850 + additional dataset	3,010	3,010
7	Remove records without usable price	3,010	2,460
8	Keep required columns + product name	2,460	2,460
9	Price normalization: values below 10,000 $\times$ 120	2,460	2,460